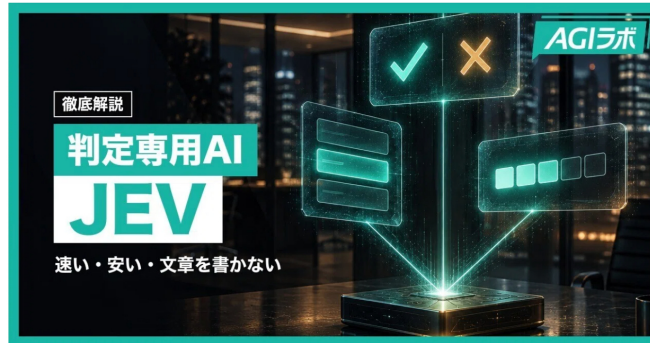




爆速で爆安。判定専用AI「JEV」を徹底解説 — 使い方・料金・実例と、実際に作った4つのプロダクトの裏側

♡ 11

AGIラボ
2026年9月18日 18:50

最近、AI界隈で「JEV (ジェブ)」というモデルが話題になっています。

♡ 11

ChatGPTやClaudeのような「文章を書くAI」とはまったく違い、JEVは **文章を一切書きません**。そのかわり、「はい／いいえ」「AとBとCのどれか」「0～4点で何点か」といった **"判定"だけを、確率つきで、非常に速く・安く** 返してくれます。

「文章を書かないAIって何に使うの？」と思うかもしれません。ところが実際に触ってみると、これが驚くほど使える。この記事では、

- JEVとは何か
- 料金
- 話題になっている活用事例
- どこから・どうやって使うか（公式API／Cloudflare）
- **実際にAGIラボで作った4つのプロダクトの裏側**（メールの返信判定、架空ペルソナ150人への導入意向調査、AIウミガメのスープ、ことばで動くRPG）

を、コード付きでまとめます。

1. JEVとは？「文章を書かない」判定専用のAI

JEVは、米TypeSafe社が開発した「**System Oneモデル**」の第1弾です。

名前の由来は、ダニエル・カーネマンの『ファスト&スロー』。人間の思考には、じっくり考える「システム2」と、一瞬で直感的に判断する「システム1」がある、という話です。LLMが「システム2」的にじっくり文章を組み立てるのに対し、JEVは「**システム1＝一瞬の判断**」に特化しています。

LLMとの違い

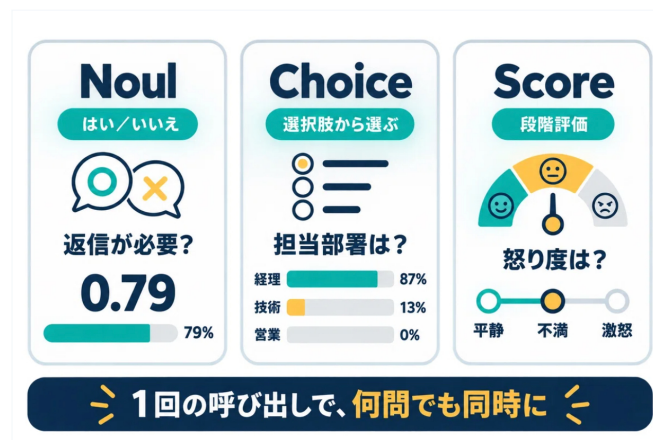
	LLM (ChatGPT・Claudeなど)	JEV
出力	文章	型の決まった答え+確率
得意	書く・考える・説明する	判定・分類・採点
速さ	数秒～	0.2～0.8秒
扱い	パースが必要	そのままコードで分岐

LLMに「このメールは返信が必要？ JSONで答えて」と頼むと、たまにJSONが壊れたり、前置きを書いたりしますよね。JEVは最初から「型の決まった答え」しか返さないで、その心配がありません。

3種類の「質問」

JEVに投げられる質問は3種類だけです。

- **Noul (ヌル)**：はい／いいえの質問。「はい」である確率を0～1で返す
 - 例：「このメールは返信が必要？」 → 0.79
- **Choice (チョイス)**：選択肢から1つ選ぶ。選んだ答えと、各選択肢の確率を返す
 - 例：「担当部署は？（経理／技術／営業）」 → 経理 87％・技術 13％・営業 0％
- **Score (スコア)**：段階評価。点数と各段階の確率を返す
 - 例：「顧客の怒り度は？（0:平静／1:不満／2:激怒）」 → 1.04



この3つを 1回の呼び出しで何個でも同時に 聞けます。質問はそれぞれ独立・並列に評価されるので、質問を増やしても応答時間はほとんど変わりません。

最大の特徴は「確率が信用できる」こと

JEVの確率は キャリブレーション（校正） されています。ざっくり言うと、「80％」と答えたものを大量に集めると、実際に約8割が当たるように訓練されている、ということです。

されている、といったことごと。

これが何を意味するかというと、**確率をそのままプログラムの判断材料に使える**ということです。

- 確信度が高い → 自動で実行
- 確信度が中くらい → ユーザーに確認
- 確信度が低い → 人間や上位のLLMに回す

LLMの「自信があります!」とは違い、JEVの「わかりません(確率が割れている)」は信用できるシグナルになります。

主なスペック(2026年9月時点)

- モデル名: jev-1.13.0 (エイリアス: jev-latest)
- 入力: テキストのみ(文字列、JSON、配列)。画像・音声・動画は非対応
- コンテキスト長: 1リクエスト64kトークン(そのうち state と最長の質問の合計で32kトークンまで)
- 言語: 英語が最も得意。**日本語も問題なく使えます**(筆者の検証では、日本語のメール判定やクイズ判定で十分な精度が出ました)

2. JEVの料金

TypeSafe公式APIの場合

- 入力100万トークンあたり **\$0.042** (約6円 ※1ドル=150円換算で6.3円)
- 出力トークンは無料
- レート制限: 1秒あたり25万トークン / 1分あたり1,200リクエスト (需要増のため変動中と公式に記載あり)



とにかく安いです。参考までに、筆者が実際に使った量で計算すると、

- メール1通の返信要否判定: 入力 約2,600トークン → 約0.016円
- 架空ペルソナ150人×6問 (計900判定): 入力 約19.5万トークン → 約 \$0.008 (約1.2円)

1通あたり0.02円以下でメールを仕分けられる計算です。

Cloudflare Workers AI経由の場合

Cloudflare Workers AIにも typesafe/jev として提供されています。Cloudflare 経由の単価は、公開ドキュメントには記載がなく、**Cloudflareのダッシュボード（AI > Models > typesafe/jev）で確認する形式**になっています（2026年9月時点）。

Cloudflareのアカウントをお持ちの方は、ダッシュボードで最新の単価を確認してから使いましょう。

3. 話題の活用事例

JEVの公開直後から、X（旧Twitter）では「速い・安い」を活かした面白いデモが次々に投稿されています。

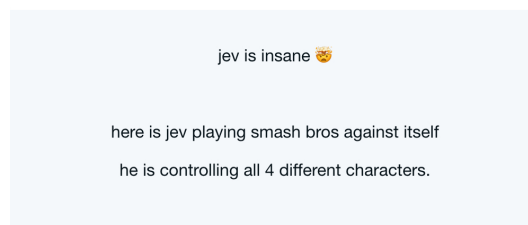
① ECのリアルタイム接客 (Rinteさん)



音声対話AI（gpt-live-1）とJEVを組み合わせたECの接客デモ。**会話の途中でも、ユーザーの発話に合わせておすすめ商品を即座に表示**します。さらに、対話内容に応じてアバターの表情を変える、という芸当も。

会話を止めずに「今の発話はどの商品カテゴリーの話か」「ユーザーの感情は？」を裏で判定し続けられるのは、JEVの速さがあってこそです。

② スマブラを4キャラ同時に操作 (Mau Baronさん)





JEVが 大乱闘スマッシュブラザーズの4キャラクターを同時に操作して、自分自身と対戦 するデモ。一瞬ごとに「今日の技を出すのがベストか」を判定しています。本人によれば、この試合で2,200万トークン以上を使ってもコストは数セント程度だったとのこと。

「gpt-6 astraの代わりにはならないが、即応性で広がる可能性は無限大」というコメントが、JEVの立ち位置をよく表しています。

③ 話した内容に合わせてスライドが自動で切り替わる (Rinteさん)



プレゼン中に 話している内容にマッチしたスライドを自動で表示。PCやポインターに触らずにページが切り替わります。「今の発話はどのスライドの話か」をChoiceで判定し続けるだけで実現できる、JEVらしい使い方です。

④ 300msごとに売買判断するトレーディングBot(Jarrold Wattsさん)

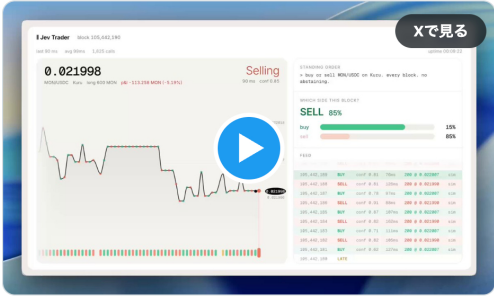
 **Jarrold Watts**  
@jarroldwatts · フォローする

I built a trading bot with Jev!

Jev decides if it should "buy" or "sell", given the price feed of an asset pair, and executes real trades.

It uses Monad to place the orders on Kuru's on-chain order book in every 300ms block.

Demo link → jev-trader.vercel.app



午前7:48 · 2026年9月17日

4,709 返信 リンクをコピー

260件の返信を読む

通貨ペアの価格フィードを見て、JEVが「買い」か「売り」かを判定し、実際に取引を執行するBot。Monadチェーン上のオンチェーン板（Kuru）に、**300msごとのブロックで注文**を出します。判断の速さがそのまま価値になる領域です。

※投資判断をAIに任せるのはリスクが伴います。あくまでデモとしてご覧ください。

⑤ 自分のX投稿3,282件を分析(Ian Nuttallさん)

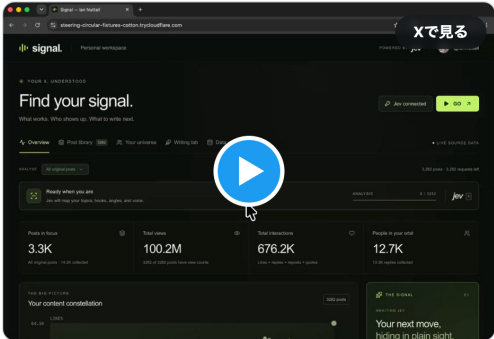
 **Ian Nuttall**  
@iannuttall · フォローする

I gave Jev 3,282 of my X posts across 100M views and asked it to find what actually works for growth.

4,252,330 tokens \$0.1282 for the full 8m 34s run!

Each post got 8 questions about the topic, hook, tone, whether it teaches something, etc.

How-to posts got 150 median likes [さらに表示](#)



午前4:31 · 2026年9月18日

605 返信 リンクをコピー

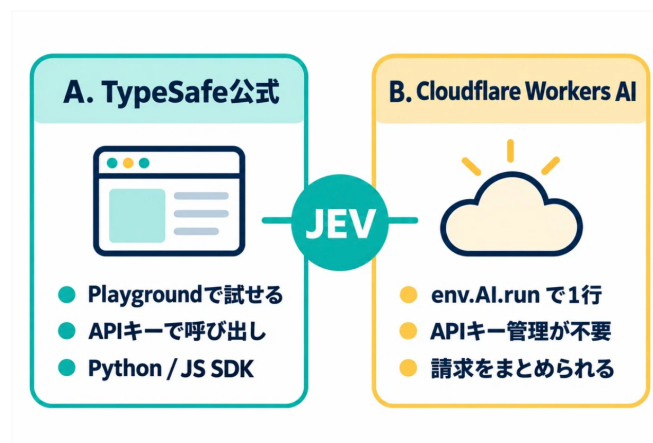
46件の返信を読む

累計1億ビューの自分のX投稿 **3,282件** をJEVに読ませ、1投稿につき8つの質問（トピック、フック、トーン、何かを教えているか、など）を判定。

- 使用トークン：約425万
- 費用：\$0.1282（約19円）
- 所要時間：8分34秒

分析の結果、「How-to系の投稿はいいね中央値150（全体の中央値は44）」
「AI×コーディングの話題は1.9倍」といった傾向が見つかったそうです。大量データへの一括判定が、とにかく安い。これもJEVの大きな強みです。

4. JEVはどこから使える？（2つの入口）



入口A: TypeSafe公式から使う

2026年9月時点では、公式APIは早期アクセス（ウェイトリスト制）です。

すぐには使えないので、まずはTypeSafeの公式サイト「Join Waitlist」から登録し、招待が届くのを待ちましょう。

1. 招待が届いたら、TypeSafeのコンソールにログイン
2. まずは **Playground**（ブラウザ上で試せる画面）で、文章と質問を入れて試す
3. 本格的に使うなら、APIキーを発行して
`https://api.typesafe.ai/v1/systemone` を呼び

公式のPython SDK・JavaScript SDKも用意されています。

向いている人：どこからでもHTTPで呼びたい人、SDKを使いたい人、レート制限やモデルのバージョンを細かく管理したい人。

入口B: Cloudflare Workers AIから使う

Cloudflare Workers AIのモデルカタログに `typesafe/jev` として登録されています。

- ドキュメント：<https://developers.cloudflare.com/ai/models/typesafe/jev/>

Cloudflare Workersの中から `env.AI.run('typesafe/jev', ...)` の1行で呼べます。
APIキーの管理が不要で、請求もCloudflareにまともります。

向いている人：すでにCloudflare Workersでサービスを作っている人、Webサービスとしてすぐ公開したい人。今回紹介するゲーム2本は、この方法で作りました。

注意点: 日本から使うときの通信の遅延(レイテンシ)

TypeSafeは「応答は70〜500ms」とうたっていますが、これは**サーバーの近くから呼んだ場合**の数字です。筆者が調べた範囲では、公式APIのサーバーは**米国西海岸（AWSのオレゴンリージョン）**にあり、日本から呼ぶと太平洋を往復する通信時間が毎回上乗せされます。

実際に日本から測ってみた結果がこちらです（2026年9月、東京）。

- 日本 ⇄ APIサーバーの往復：**約0.16秒**（接続を新しく張るときは、暗号化の準備も含めて**約0.33秒**）
- Cloudflare Workers（東京）経由でJEVを1回呼ぶ：**0.19〜0.85秒**（中央値約0.33秒）
- 公式APIを直接呼ぶ：**0.16〜0.44秒**
- ごくまれに、1回だけ数秒〜数十秒かかることもありました

メールの仕分け、チャットの返答、クイズの判定のような用途なら**体感はほぼ「一瞬」**で、まったく問題ありません。一方で、スマブラのデモのように**1秒に何回も判断させる**使い方や、数百ミリ秒を争う取引では、この往復時間が効いてきます。

日本から使うときのコツ

- **質問は1回の呼び出しにまとめる**：JEVは質問を増やしても時間がほとんど変わらないので、往復の回数を減らすのが一番効く
- **並列で投げる**：ペルソナ150人の調査は、20並列で900判定を約5秒で終わっています
- **接続を使い回す**：毎回新しく接続すると、そのたびに約0.3秒余計にかかる
- **先回りして聞いておく**：次に必要になりそうな判定を、ユーザーの操作を待たずに投げておく（公式ドキュメントでも「Speculative fan-out」として紹介されている考え方）
- **待ち時間の上限と代わりの手を決めておく**：メール判定では、JEVが失敗したらLLMに切り替えるようにしました。ゲームでは「考え中…」の表示を出すだけでも体験がかなり変わります
- **本当にリアルタイム性が必要ななら、処理をするサーバーを米国に置く**：日本のユーザーには結果だけを返す構成にすれば、判断の往復は米国内で完結します

5. 具体的な使い方

基本の形：「state(判断材料)」と「questions(質問)」を渡すだけ

公式APIの場合（JavaScript）：

```
const res = await fetch('https://api.typesafe.ai/v1/systemone', {
  method: 'POST',
  headers: {
    'Authorization': `Bearer ${process.env.TYPESAFE_API_KEY}`,
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({
    model: 'jev-latest',
    // 判断材料。文字列でもJSONでもOK
    state: '決済が3日間ずっと失敗しています！至急対応してください。',
    questions: {
      is_urgent: {
        type: 'noul',
        instructions: 'このメッセージは緊急性を伝えていますか？',
      },
      department: {
        type: 'choice',
        instructions: 'どのチームが対応すべきですか？',
        criteria: {
          billing: '支払い・請求・返金',
          technical: '不具合・障害・連携',
          sales: '料金プラン・新規契約',
        },
      },
      frustration: {
        type: 'score',
        instructions: '顧客はどのくらい苛立っていますか？',
        criteria: ['落ち着いている', '不満がある', '激怒している'],
      },
    },
  }),
})
const { answers } = await res.json()
```

返ってくる答え（イメージ）：

```
{
  "model": "jev-1.13.0",
  "answers": {
    "is_urgent": { "type": "noul", "noul": 0.95 },
    "department": {
      "type": "choice",
      "choice": "billing",
      "confidence": 0.8,
      "probabilities": {
        "billing": 0.87,
        "technical": 0.13,
        "sales": 0
      }
    },
    "frustration": {
      "type": "score",
      "score": 1.04,
      "confidence": 0.94,
      "probabilities": { "0": 0, "1": 0.96, "2": 0.04 }
    }
  },
  "usage": { "input_tokens": 426, "output_tokens": 73 }
}
```

あとは if (answers.is_urgent.noul > 0.8) のように、普通のプログラムとして分岐するだけです。

Cloudflare Workersの場合

wrangler.jsonc にAIバインディングを追加して、

```
{
  "name": "my-app",
  "main": "src/index.ts",
  "ai": { "binding": "AI" }
}
```

Workerのコードから呼び出します。

```
const r = await env.AI.run('typesafe/jev', { state, questions })
// Cloudflare経由では結果が result に包まれて返ってきた
// { state: "Completed", result: { model, answers, usage } }
const { answers } = r.result ?? r
```

筆者の環境では、Cloudflare経由だとレスポンスが result で包まれて返ってきたので、上のように両対応にしておくと安全です。

使いこなしのコツ(実際に作って分かったこと)

① 質問は「一瞬で答えられる粒度」に分解する

「このスタートアップのピッチを評価して」ではなく、「市場規模は?」「技術的に実現可能?」「差別化されている?」と分けて聞き、点数の合成はコード側でやる。公式ドキュメントでも推奨されている考え方です。

② 選択肢の説明文は、その場で動的に作ってよい

RPGでは、選択肢を「赤いスライム。主人公から見て右上に3マス」のように、**その時点のマップの状況から毎回生成**しました。これだけで「右上のスライムを倒して」が正しく解釈できます。

③ JEVは「作る」のは苦手、「選ぶ」のは得意

JEVは文章も計画も生成できません。でも、候補をプログラムで作って**JEVに選ばせる**と、生成AIのようなことができます（後述のRPGで詳しく）。

④ 確率・確信度をUXや業務ルールに使う

「確信度70%未満なら人間に回す」「はい58%のように確率ごとに見せる」など、確率そのものが価値になります。

6. 実際に作った4つのプロダクトの裏側

ここからは、AGIラボで実際にJEVを使って作ったものを紹介します。

1. **メールの返信要否判定**（業務で毎時稼働中）
2. **架空ペルソナ150人への導入意向調査**
3. **AIウミガメのスープ**（Webゲーム）
4. **ことばクエスト**（言葉だけで操作するRPG）

6-1. メール返信要否判定 — LLMからJEVに置き換えた

何をしているか

筆者のGmailでは、1時間ごとに自動で動くプログラム（AGI Wingsというサーバーレス基盤上のハンドラー）が、新着メールを仕分けています。

1. 新着スレッドを取得
2. 「自分が返信すべきメールか」を判定
3. 「AI/返信要」「AI/返信不要」のラベルを付ける
4. 返信要のメールには、返信の下書きをLLMで作成しておく

朝メールを開くと、返信すべきものだけに下書きが用意されている、という仕組みです。



このうち 2. の判定を、LLM（OpenRouter経由のgpt-5.4-mini）からJEVに置き換えました。下書きの作成は文章の生成なので、LLMのままです。

なぜJEVに置き換えたか

- 判定は「生成」ではなく「分類」だから：LLMにJSONで答えさせてパースする方式は、ときどきパースに失敗していました
- 速くて安い：判定だけなら、JEVで十分
- 確率が取れる：「返信要 79%」のように、どれくらい迷ったかが分かる

前処理はプログラムで

JEVに聞く前に、明らかなものはプログラムで弾いています。

- 最後のメッセージが自分の送信 → スキップ
- 送信元に noreply を含む → 返信不要
- To/Ccに自分が含まれていない → 返信不要

AIに聞かなくていいことは聞かない。これはLLMでもJEVでも同じ鉄則です。

JEVに渡す「state」の設計

```
const state = {
  recipient: '私は石川陽太です。必ず私の立場で判断してください。',
  recipient_role: 'CC のみで受信（直接の宛先ではない）', // または「直接の宛先(To)」
  reply_criteria: `1. お知らせやメルマガには返信不要
2. 質問や依頼を含むメールには返信が必要
3. 個人的なメッセージや重要な業務連絡には返信が必要
4. 自動生成されたシステムメッセージには通常返信不要
`,
  thread: threadContext, // スレッド全体(古い順、最大8,000文字)
  latest_message: { from, to, cc, subject, date, body }, // 判定対象の最新メッセージ
}
```

ポイントは、LLM版で使っていた「判断基準」の文章を、そのまま state に入れている ことです。JEVのstateはJSONで構造化できるので、「受信者の情報」「判断基準」「スレッド」「最新メール」を分けて渡すと、質問文の中で「reply_criteria に従って」「latest_message について」と名前で参照できます。

質問は2つを1回で

```
const questions = {  
  // 本題:返信が必要か(NouI)  
  needs_reply: {  
    type: 'nouI',
```